INVENTORY MANAGEMENT SYSTEM - TECHNICAL REPORT

TABLE OF CONTENTS

DELIVERABLE 1: DATA MODELING REPORT

1.1 INTRODUCTION

The Inventory Management System is designed to support organizations in managing customers, suppliers, items, warehouses, sales transactions, and purchase operations. This report documents the data modeling process behind the system, beginning with the conceptual understanding of entities and progressing toward the logical structure and supporting business rules. Our goal is to provide a clear and coherent representation of how the system manages inventory activities and ensures data integrity.

1.2 CONCEPTUAL DATA MODEL

The conceptual model captures the essential real world entities and illustrates how they interact within the inventory environment. It includes customers, suppliers, items, categories, admin users, warehouses, and the transactional components of sales and purchases.

The model demonstrates key operational flows: items are acquired from suppliers through purchase transactions, stored within warehouses as stock, and later sold to customers through sales. This high level representation identifies all major actors and resources without considering database constraints, serving as the foundation for more detailed models.

1.3 LOGICAL DATA MODEL

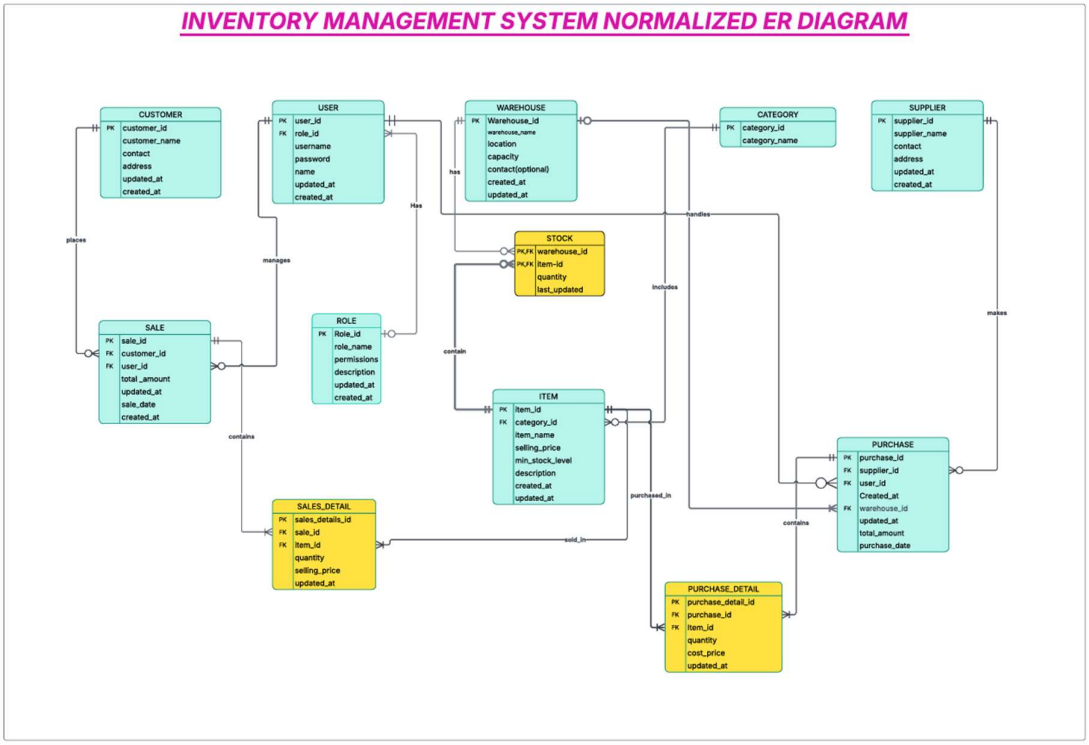
The logical model takes the conceptual relationships and ERD structure and converts them into formal entities with attributes and foreign key constraints. Every entity now includes a primary key, and relationships are implemented through foreign keys.

Master records define stable information about customers, suppliers, categories, items, and warehouses. Transaction entities capture time-based data (sales and purchases). Detail tables store item-level specifics, while the stock table maintains real-time inventory availability.

The logical model ensures that redundancy is minimum and relationships are preserved according to the business rules defined in the conceptual model.

1.4 ERD (ENTITY-RELATIONSHIP DIAGRAM) EXPLANATION

The ERD provides a structured visual representation of the system's data entities and the relationships between them. Below is a detailed explanation of the diagram based on the conceptual model.



1.4.1 Major Entities

The ERD consists of the following primary entities:

1. **Customer** - Represents individuals or organizations that buy items from the system
2. **Supplier** - Represents providers that deliver items through purchase orders
3. **Item** - The core product involved in both sales and purchases
4. **Category** - Groups items into logical classifications
5. **Warehouse** - Physical storage locations where items are stocked
6. **Admin** - System users responsible for managing and performing operational tasks
7. **Sale** - Represents customer transactions
8. **Sales Detail** - Stores item-level details of each sale
9. **Purchase** - Represents a procurement transaction from a supplier
10. **Purchase Detail** - Stores item-level details of each purchase
11. **Stock** - Represents the current quantity of items available in each warehouse
12. **Role** - Represents that each admin has a role

1.4.2 Key Relationships in the ERD

Customer → Sale

- A customer can place multiple sales, but each sale belongs to exactly one customer

Sale → Sales Detail → Item

- A sale may consist of multiple items
- Sales detail breaks down item quantity and price
- Each entry links to a single item

Supplier → Purchase

- A supplier may make many purchase transactions delivered to the organization

Purchase → Purchase Detail → Item

- Similar to sales, a purchase contains multiple items
- Each purchase detail record describes the item quantity and cost

Warehouse → Stock → Item

- Warehouses can store multiple items, and items may be held across multiple warehouses
- Stock resolves this many-to-many relationship by storing the current quantity of each item per location

Category → Item

- Items are grouped under categories to organize the inventory
- Each item belongs to exactly one category

Admin → All Managed Entities

- Admins oversee the creation and modification of records such as customers, suppliers, items, and warehouses
- This reflects the managerial control component of the system

User → Role

- Shows that the user will have a role

1.4.3 ERD Structure Summary

The ERD clearly separates **master data** (customers, suppliers, items, warehouses, categories) from **transactional data** (sales, purchases, and their details). It also introduces a **supporting entity** (stock) that ensures accurate inventory tracking at all locations.

This structure ensures that every real-world process—buying, storing, and selling—has a corresponding, integrity-enforced representation in the database.

1.5 RELATIONAL SCHEMA

The relational schema organizes the system into normalized tables, identifying primary keys and foreign keys. Master tables maintain reference information; transaction tables capture sales and purchases; detail tables break each transaction into item-level components; and the stock table handles warehouse-item inventory levels.

This schema establishes data integrity and efficient structure for querying and reporting, directly reflecting the logical model.

-- Core Tables (12 total):

1. ROLE (role_id, role_name, permissions, description)

2. USER_TABLE (user_id, role_id, username, password, name)

3. CUSTOMER (customer_id, customer_name, contact, address)

4. SUPPLIER (supplier_id, supplier_name, contact, address)

5. CATEGORY (category_id, category_name)

6. WAREHOUSE (warehouse_id, warehouse_name, location, capacity)

7. ITEM (item_id, category_id, item_name, selling_price, min_stock_level)

8. PURCHASE (purchase_id, supplier_id, user_id, warehouse_id, total_amount)

9. PURCHASE_DETAIL (purchase_detail_id, purchase_id, item_id, quantity, price)

10. SALE (sale_id, customer_id, user_id, total_amount, sale_date)

11. SALES_DETAIL (sales_details_id, sale_id, item_id, quantity, selling_price)

12. STOCK (warehouse_id, item_id, quantity) -- Composite PK

1.6 FUNCTIONAL REQUIREMENTS

The Inventory System must:

- Allow administrators to manage all master data
- Record, track, and update purchase and sales transactions
- Automatically update stock quantities in real time
- Provide reporting and data visualization for inventory performance
- Validate entries to ensure accurate and consistent data
- Notify users when stock levels are critically low
- Maintain user authentication to control access to operations

These requirements define what the system must accomplish from the user's perspective.

1.7 NON-FUNCTIONAL REQUIREMENTS

The system must exhibit high reliability, secure user authentication, fast response time, and ease of use. It must support scalability to handle growing data volumes and maintain strict data integrity. Backup and recovery mechanisms protect the system from data loss.

These requirements ensure that the system performs efficiently and safely in practical settings.

1.8 BUSINESS RULES

- Each sale must have at least one sales detail entry
- Each purchase must include at least one purchase detail
- An item cannot belong to multiple categories
- Stock quantities must always remain non-negative
- A warehouse cannot hold stock for non-existent items
- Customers may have multiple sales but cannot have sales without customer records
- Suppliers must exist before making purchases
- Admin credentials must be unique and secure
- Admin will have a role

These rules govern how data behaves inside the system.

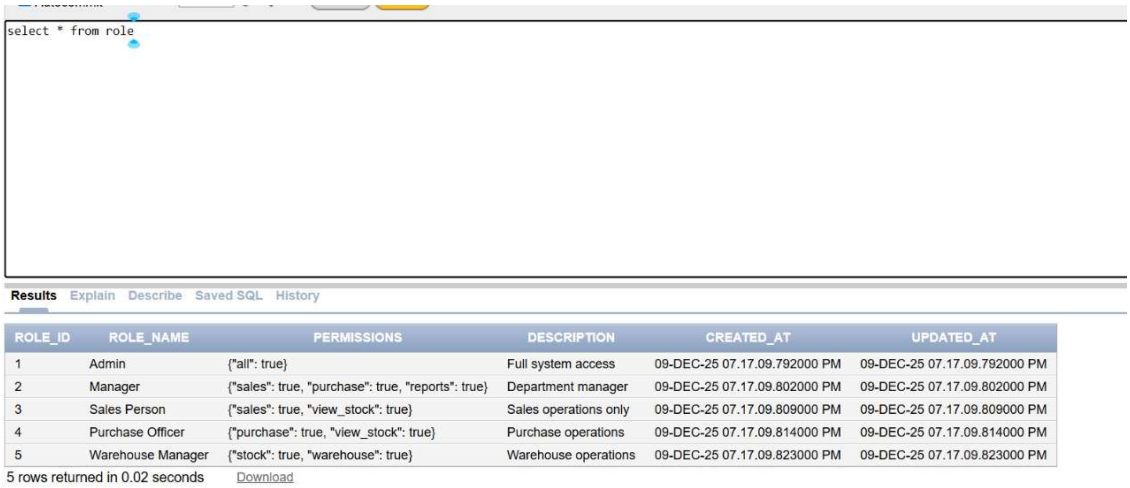
1.9 CONCLUSION

This deliverable presents a structured overview of the Inventory Management System from conceptual understanding to logical implementation. The ERD provides a clear visualization of how all entities interact, and the logical model operates these relationships inside a database. Together with the defined functional requirements, non-functional requirements, and business rules, this modeling process ensures a complete and efficient foundation for building a robust inventory system.

DELIVERABLE 2: DATABASE IMPLEMENTATION

2.1 DDL (DATA DEFINITION LANGUAGE)

- Defines the **complete database structure**.
- Includes creation of:
    - All **tables**
    - **Primary keys**
    - **Foreign keys**
    - **Unique, NOT NULL, and CHECK constraints**
- Tables use **appropriate data types** for each attribute.
- **Sequences** implemented to auto-generate primary key values.
- Follows the **normalized logical model** designed earlier.
- Enforces **business rules** through:
    - Constraints
    - Referential integrity
- Ensures **consistency** between tables using foreign key relationships.
- Guarantees structural correctness of the entire database.

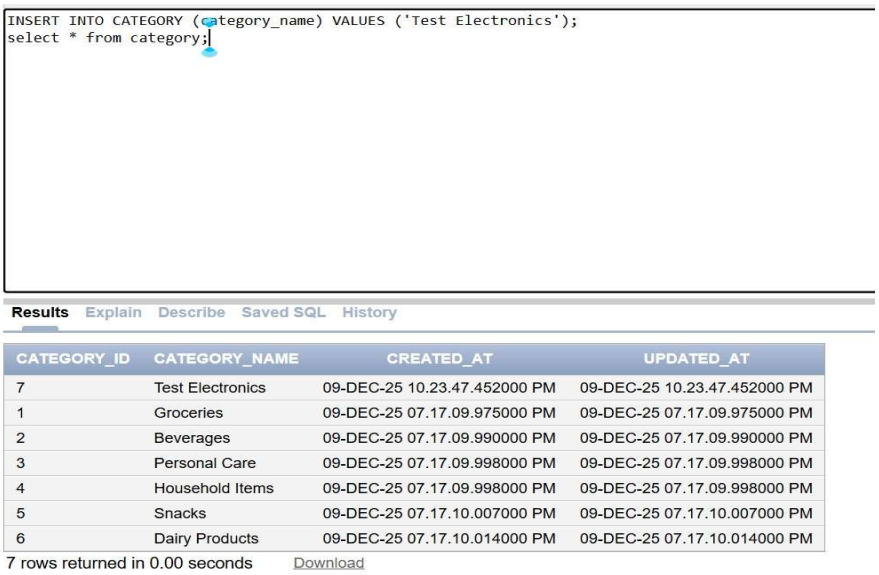


2.2 DML (DATA MANIPULATION LANGUAGE)

- Inserts **test data** into all tables.
- Dataset includes:
    - **5 roles**
    - **7 users**
    - **5 customers**
    - **5 suppliers**
    - **6 categories**
    - **3 warehouses**
    - **10 items**
    - **4 purchases** with **9 purchase details**
    - **5 sales** with **8 sales details**
- Provides **realistic sample data** for end-to-end testing.
- Enables full testing of:
    - Master data management
    - Transaction processing

- Inventory tracking

- Inserts data in **correct dependency order** to satisfy foreign key constraints.
- Ensures smooth validation of overall system functionality.



```
INSERT INTO CATEGORY (category_name) VALUES ('Test Electronics');
select * from category;
```

Results Explain Describe Saved SQL History

CATEGORY_ID	CATEGORY_NAME	CREATED_AT	UPDATED_AT
7	Test Electronics	09-DEC-25 10.23.47.452000 PM	09-DEC-25 10.23.47.452000 PM
1	Groceries	09-DEC-25 07.17.09.975000 PM	09-DEC-25 07.17.09.975000 PM
2	Beverages	09-DEC-25 07.17.09.990000 PM	09-DEC-25 07.17.09.990000 PM
3	Personal Care	09-DEC-25 07.17.09.998000 PM	09-DEC-25 07.17.09.998000 PM
4	Household Items	09-DEC-25 07.17.09.998000 PM	09-DEC-25 07.17.09.998000 PM
5	Snacks	09-DEC-25 07.17.10.007000 PM	09-DEC-25 07.17.10.007000 PM
6	Dairy Products	09-DEC-25 07.17.10.014000 PM	09-DEC-25 07.17.10.014000 PM

7 rows returned in 0.00 seconds Download

2.3 NORMALIZATION ANALYSIS

2.3.1 Introduction

Normalization is a systematic process used to organize data efficiently, eliminate redundancy, and maintain data integrity. This section analyzes the database through First (1NF), Second (2NF), and Third Normal Forms (3NF). Each normal form is examined to ensure the database meets the required criteria, eliminates anomalies, and supports efficient data querying and updating.

2.3.2 First Normal Form (1NF)

1NF Requirements: A table is in 1NF if:

1. All values are atomic (indivisible)
2. There are no repeating groups or multi-valued attributes
3. Each record is unique (primary key exists)

Schema in 1NF:

The database consists of the following relations:

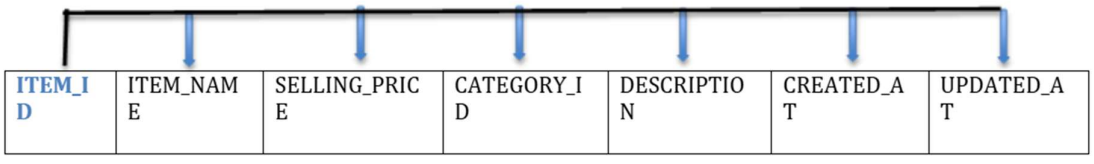
- SUPPLIER (supplier_id, supplier_name, contact, address, updated_at)
- CUSTOMER (customer_id, customer_name, contact, address, updated_at)
- USER (user_id, username, password, user_name, updated_at)
- CATEGORY (category_id, category_name)
- ITEM (item_id, item_name, selling_price, category_id, min_stock_level, description, created_at, updated_at)
- WAREHOUSE (warehouse_id, warehouse_name, location, capacity, contact, created_at, updated_at)
- STOCK (warehouse_id, item_id, quantity, last_updated)
- PURCHASE (purchase_id, purchase_date, supplier_id, total_amount, created_at, updated_at)
- PURCHASE_DETAIL (purchase_detail_id, purchase_id, item_id, quantity, cost_price, updated_at, created_at)
- SALE (sale_id, sale_date, customer_id, total_amount, created_at, updated_at)
- SALES_DETAIL (sales_detail_id, sale_id, item_id, quantity, selling_price, updated_at)
- ROLE (role_id, role_name, permissions, description, created_at, updated_at)

Why the Database Satisfies 1NF:

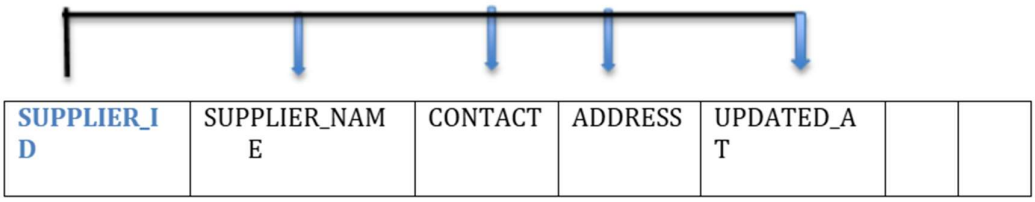
- **Atomic values**: All fields store single values such as names, prices, dates, and contacts
- **No repeating groups**: Purchase and sale items are stored in separate detail tables, avoiding repetition inside PURCHASE or SALE
- **Unique rows**: Each table has a primary key ensuring uniqueness
- **Single-valued attributes**: Stock quantities for each warehouse-item combination are stored in separate rows, ensuring clean, atomic data storage

Functional Dependencies:

- supplier_id → supplier_name, contact, address, updated_at



- customer_id → customer_name, contact, address, updated_at

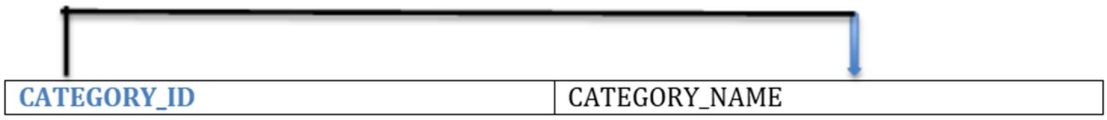


SUPPLIER_ID	SUPPLIER_NAME	CONTACT	ADDRESS	UPDATED_AT		

- admin_id → username, password, admin_name, updated_at

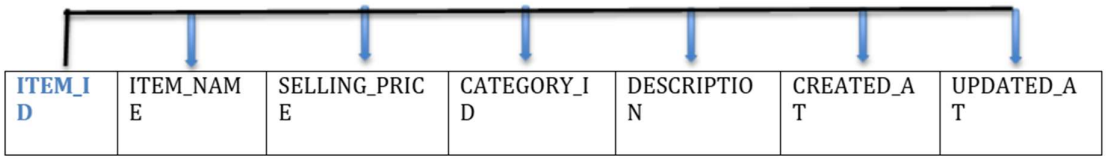
ADMIN_ID	USERNAME	PASSWORD	ADMIN_NAME	UPDATED_AT

- category_id → category_name



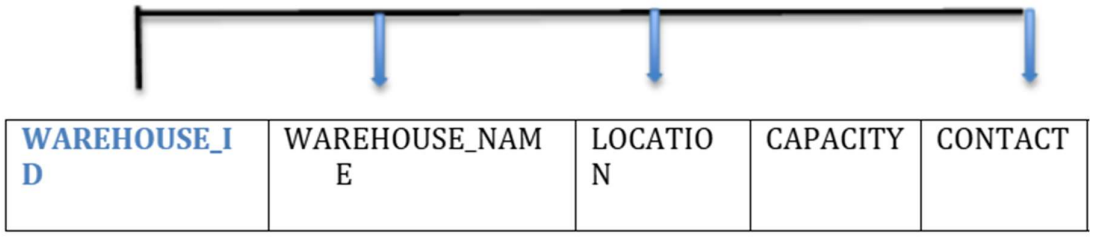
CATEGORY_ID	CATEGORY_NAME

- item_id → item_name, selling_price, category_id, min_stock_level, description, created_at, updated_at



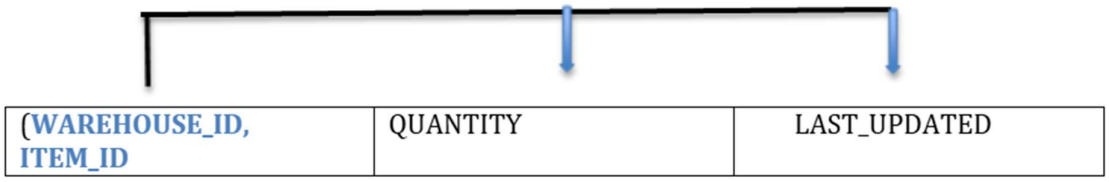
ITEM_ID	ITEM_NAME	SELLING_PRICE	CATEGORY_ID	DESCRIPTION	CREATED_AT	UPDATED_AT

- warehouse_id → warehouse_name, location, capacity, contact

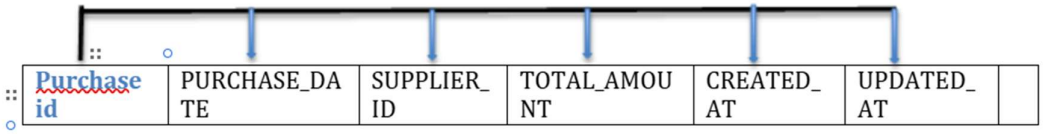


WAREHOUSE_ID	WAREHOUSE_NAME	LOCATION	CAPACITY	CONTACT

- (warehouse_id, item_id) → quantity, last_updated

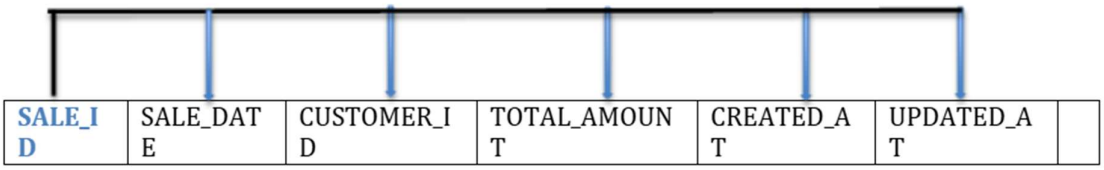


- purchase_id → purchase_date, supplier_id, total_amount, created_at, updated_at

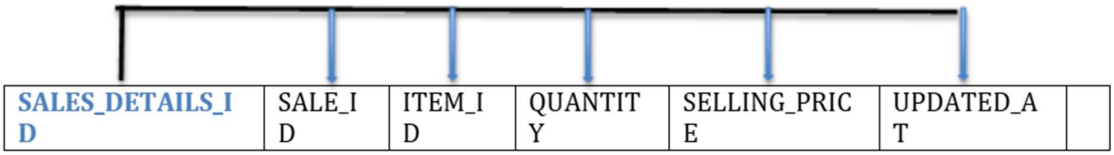


- purchase_detail_id → purchase_id, item_id, quantity, cost_price, updated_at

- sale_id → sale_date, customer_id, total_amount, created_at, updated_at



- sales_detail_id → sale_id, item_id, quantity, selling_price, updated_at



These dependencies show that each non-key attribute is fully determined by the primary key of its table.

2.3.3 Second Normal Form (2NF)

2NF Requirements: A table is in 2NF if:

1. It is already in 1NF

2. It has no partial dependencies—no non-key attribute should depend on a part of a composite key

Analysis of 2NF Compliance:

Most tables such as SUPPLIER, CUSTOMER, ADMIN, CATEGORY, ITEM, PURCHASE, SALE, PURCHASE_DETAIL, and SALES_DETAIL have single-column primary keys, meaning partial dependencies cannot exist.

The only table with a composite primary key is:

- **STOCK (warehouse_id, item_id, quantity, last_updated)**

In this case:

- quantity depends on **both** warehouse_id AND item_id
- last_updated also depends on the combined key
- There are **no attributes** depending solely on warehouse_id or item_id

Conclusion for 2NF:

- No partial dependencies exist within any table
- The database structure already satisfies 2NF without requiring modification

2.3.4 Third Normal Form (3NF)

3NF Requirements: A table is in 3NF if:

1. It is already in 2NF
2. It has no transitive dependencies—a non-key attribute should not determine another non-key attribute

Analysis of 3NF Compliance:

- **SUPPLIER & CUSTOMER**: Attributes such as contact and address depend only on their respective IDs; no attribute determines another
- **ITEM**: Attributes like item_name, selling_price, and min_stock_level depend solely on item_id. category_id is a foreign key, not a derived attribute
- **WAREHOUSE**: All information (name, location, capacity, contact) is directly dependent on warehouse_id
- **PURCHASE & SALE**: Attributes such as total_amount, dates, and references to supplier or customer depend only on the primary key

- **DETAIL TABLES**: purchase_detail_id and sales_detail_id determine all their associated attributes with no transitive dependencies

Importantly, historical values (e.g., cost_price and selling_price during transactions) are stored correctly to avoid dependency on ITEM's current price, ensuring consistency and preserving historical data.

Conclusion for 3NF:

- There are **no transitive dependencies**
- The database meets all requirements for 3NF
- No restructuring is needed to achieve 3NF

2.3.5 Database Keys and Relationships

Primary Keys: Each table includes a well-defined primary key, ensuring uniqueness.

Foreign Keys: Foreign keys link tables to maintain referential integrity. Examples include:

- PURCHASE.supplier_id → SUPPLIER
- ITEM.category_id → CATEGORY
- SALES_DETAIL.sale_id → SALE
- STOCK.item_id → ITEM

These constraints enforce consistency and link related entities.

2.3.6 Benefits of the Normalized Design

1. Reduced Redundancy Data is stored in appropriate tables without unnecessary duplication.

2. Improved Data Integrity Primary and foreign keys ensure consistent and reliable relationships between records.

3. Efficient Data Retrieval Normalized structures reduce search and update anomalies, improving reliability.

4. Historical Data Preservation Purchase and sale detail tables preserve actual historical cost and selling prices, ensuring accurate reporting.

5. Multi-Warehouse Support The STOCK table design supports tracking inventory across multiple warehouses without redundancy.

2.3.7 Normalization Conclusion

The system successfully achieves **First, Second, and Third Normal Forms** through careful design, appropriate table decomposition, and correct application of primary and foreign keys. No normalization violations were found, and the structure is optimized for efficiency, scalability, and long-term integrity.

The result is a reliable, well-organized database suitable for inventory, purchase, and sales management across multiple warehouses.

2.4 STORED LOGIC

2.4.1 Stored Procedures

The system implements three advanced stored procedures for comprehensive business process automation:

#	Object Type	Name	Purpose	Complexity
1	Function	`CENTER`	Text alignment utility for formatted reports	Low
2	Procedure	`PROC_MONTHLY_BUSINESS_PROCESSING`	Generate monthly financial reports (pre-existing)	Medium
3	Procedure	`PROC_PROCESS_BULK_PURCHASE`	Process multiple item purchases in single transaction	High
4	Procedure	`PROC_ANALYZE_INVENTORY_PERFORMANCE`	Comprehensive inventory analytics and recommendations	High

1. PROC_PROCESS_BULK_PURCHASE

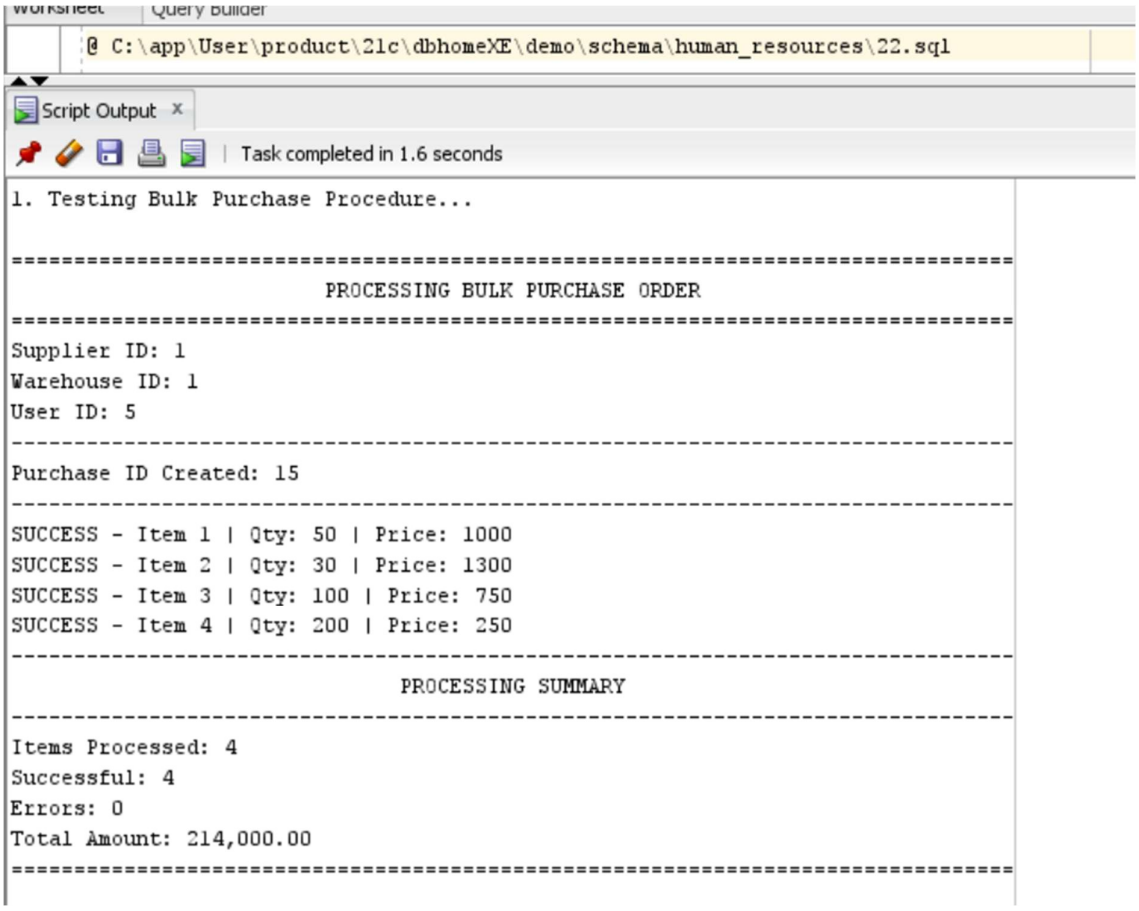
Business Logic:

- Parses comma-separated item lists (format: item_id:quantity:price)
- Creates purchase header and detail records
- Validates each item exists before processing
- Implements transaction control with savepoints

- Provides real-time processing feedback

Key Features:

- **Transaction Management:** Uses savepoints for rollback capability
- **Error Handling:** Individual item errors don't fail entire batch
- **Validation:** Checks item existence before insertion
- **Audit Trail:** Tracks success/error counts per transaction



2. PROC_ANALYZE_INVENTORY_PERFORMANCE

Business Logic:

- Analyzes sales data over specified date range (default: last 90 days)
- Calculates stock turnover ratios
- Identifies high-demand items

- Flags items needing reorder
- Provides actionable business recommendations

Key Features:

- **Multi-cursor Analysis:** Uses 2 cursors for different analytical views
- **Dynamic Calculations:** Computes turnover ratios, averages, totals
- **Business Intelligence:** Automatic status classification (REORDER NOW, MONITOR, HEALTHY)
- **Recommendations Engine:** Generates alerts based on inventory health

```
================================================================================
                    INVENTORY PERFORMANCE ANALYSIS REPORT
================================================================================
Period: 01-DEC-2024 to 31-DEC-2024
Report Date: 10-DEC-2025 20:20:09
--------------------------------------------------------------------------------
                             TOP 5 SELLING ITEMS
--------------------------------------------------------------------------------
Item                          Units Sold  Revenue        Avg Price
--------------------------------------------------------------------------------
Tapal Tea 950g                80          68,000         850.
Basmati Rice 5kg              50          60,000         1,200.
Cooking Oil 5L                30          45,000         1,500.
Nestle Milk Pack 1L           50          14,000         280.
Lays Family Pack              60          10,800         180.
================================================================================
                           STOCK TURNOVER ANALYSIS
--------------------------------------------------------------------------------
Item                          Current Stock  Units Sold  Turnover  Status
--------------------------------------------------------------------------------
Olpers Cream 200ml            70             30          0.43      REORDER NOW
Lux Soap 3pk                  160            40          0.25      MONITOR
Lays Family Pack              240            60          0.25      MONITOR
Kolson Noodles Box            80             20          0.25      REORDER NOW
Basmati Rice 5kg              420            50          0.12      HEALTHY
```

```
Basmati Rice 5kg          420          50          0.12        HEALTHY
Cooking Oil 5L            260          30          0.12        HEALTHY
Tapal Tea 950g            786          80          0.1         HEALTHY
Nestle Milk Pack 1L       1,450        50          0.03        HEALTHY
Surf Excel 3kg            0            0           0.          REORDER NOW
Fair 22 50g               150          0           0.          HEALTHY
================================================================================
                              SUMMARY STATISTICS
--------------------------------------------------------------------------------
Total Items Analyzed: 10
High Demand Items (Turnover > 2): 0
Items Needing Reorder: 3
Total Revenue in Period: 197,800
Avg Revenue per Item: 19,780
================================================================================
                              RECOMMENDATIONS
--------------------------------------------------------------------------------
ALERT: 3 items need immediate reordering
NOTE: Stock levels appear adequate for current demand
================================================================================


PL/SQL procedure successfully completed.
```

. 3. CENTER Function

Purpose: Text alignment utility for formatted reports
Feature: Handles odd/even string lengths correctly

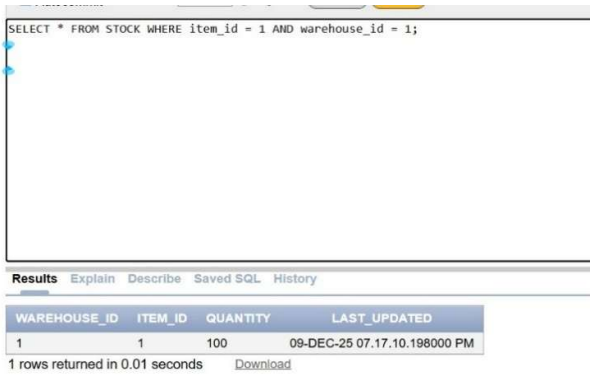
2.4.2 Triggers

Database triggers are implemented to automate critical business logic and maintain data consistency:

Stock Update Trigger: Automatically updates stock quantities when purchase details are inserted, ensuring real-time inventory tracking without manual intervention.

Working:

First the amount of stock is:



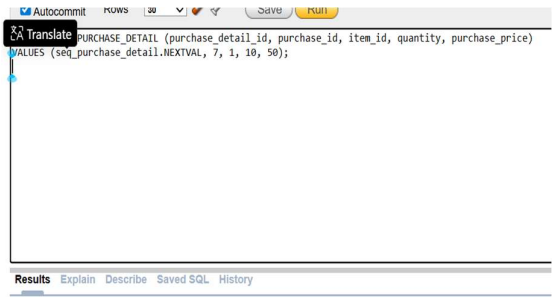
After item is purchased:

```
INSERT INTO PURCHASE(purchase_id, supplier_id, warehouse_id, total_amount,purchase_date,user_id)
VALUES (seq_purchase.NEXTVAL, 1, 1,10, SYSDATE,1);
```

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.11 seconds



It is added to purchase detail

```
select * from stock where warehouse_id=1 and item_id=1;
```

Results Explain Describe Saved SQL History

WAREHOUSE_ID	ITEM_ID	QUANTITY	LAST_UPDATED
1	1	110	10-DEC-25 05.27.34.619000 PM

1 rows returned in 0.01 seconds Download

Now the quantity has been updated from 100+10=110 showing auto increment

Renew check:

Auto_increment: category:

Before:

```
INSERT INTO CATEGORY (category_name) VALUES ('Test Electronics');
```

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.03 seconds

Shows auto increment: Ensure category auto-filled from sequence when not provided

after

```
select * from category;
```

Results Explain Describe Saved SQL History

CATEGORY_ID	CATEGORY_NAME	CREATED_AT	UPDATED_AT
7	Test Electronics	09-DEC-25 10.23.47.452000 PM	09-DEC-25 10.23.47.452000 PM
1	Groceries	09-DEC-25 07.17.09.975000 PM	09-DEC-25 07.17.09.975000
2	Beverages	09-DEC-25 07.17.09.990000 PM	09-DEC-25 07.17.09.9900
3	Personal Care	09-DEC-25 07.17.09.998000 PM	09-DEC-25 07.17.09.998000 PM
4	Household Items	09-DEC-25 07.17.09.998000 PM	09-DEC-25 07.17.09.998000 PM
5	Snacks	09-DEC-25 07.17.10.007000 PM	09-DEC-25 07.17.10.007000 PM
6	Dairy Products	09-DEC-25 07.17.10.014000 PM	09-DEC-25 07.17.10.014000 PM

7 rows returned in 0.00 seconds Download

Auto_increment time stamp:

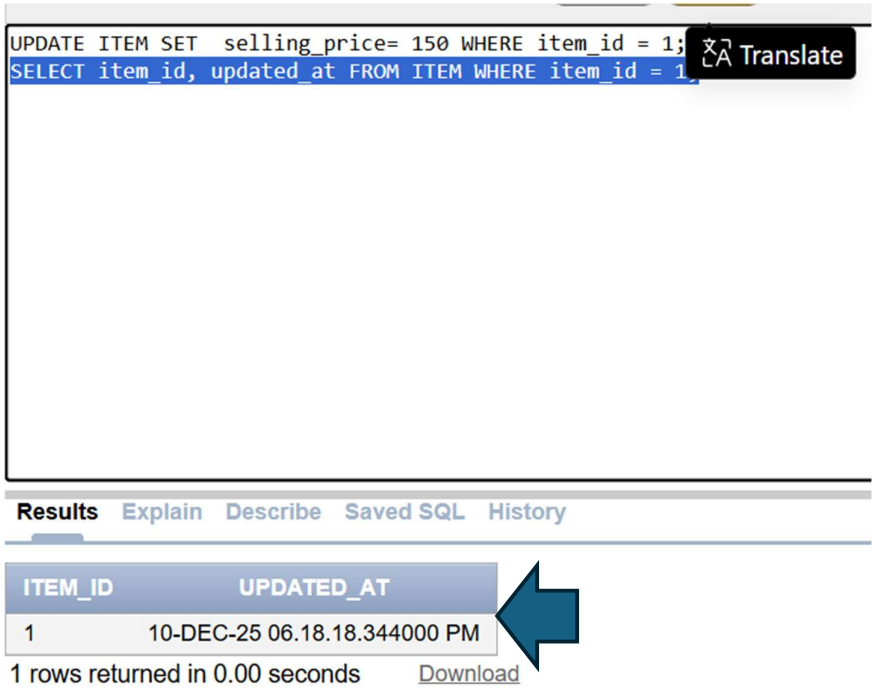
Before:

```
SELECT item_id, updated_at FROM ITEM WHERE item_id = 1;
```

Results Explain Describe Saved SQL History

ITEM_ID	UPDATED_AT
1	09-DEC-25 10.30.30.741000 PM

After:



Sales Stock Deduction Trigger: Automatically decreases stock levels when sales are recorded, preventing overselling and maintaining accurate inventory counts.

SHOWING trigger of auto-decrement:

A sale is made:

Sale details are added: trigger shows item is sold from warehouse:

Quantity is reduced from 110-2(sale)=108

Hence it shows the 3 types of triggers:

Auto-increment primary key

Auto-update of timestamp after changes are made

Sale and purchase stock management triggers.

```
select * from stock where warehouse_id=1 and item_id=1;
```

Results Explain Describe Saved SQL History

WAREHOUSE_ID	ITEM_ID	QUANTITY	LAST_UPDATED
1	1	110	10-DEC-25 05.27.34.619000 PM

1 rows returned in 0.01 seconds Download

A sale is made:

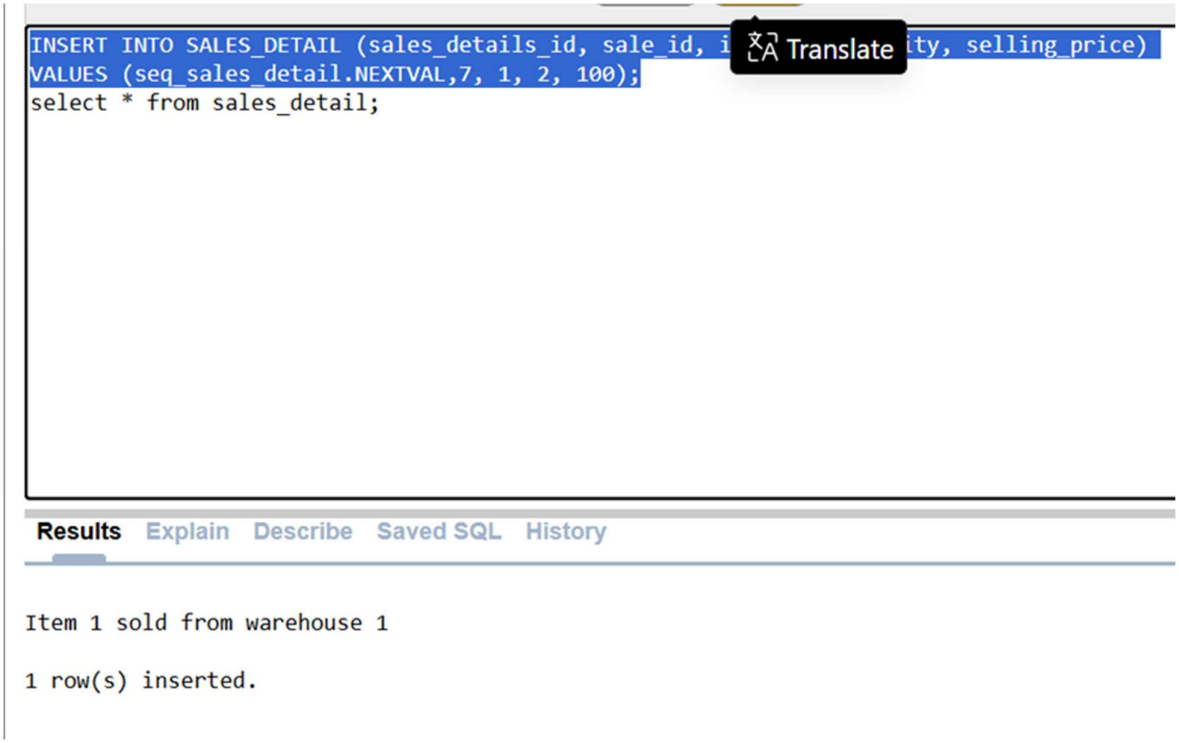
```
INSERT INTO SALE (sale_id, customer_id, sale_date,user_id,total_amount)
VALUES (seq_sale.NEXTVAL, 1, SYSDATE,3,2);
```

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.01 seconds

Sale details are added: trigger shows item is sold from warehouse:



Quantity is reduced from 110-2(sale)=108

Hence it shows the 3 types of triggers:

Auto-increment primary key

Auto-update of timestamp after changes are made

Sale and purchase stock management triggers.

```
SELECT * FROM STOCK WHERE item_id = 1 AND warehouse_id = 1;
```

Results Explain Describe Saved SQL History

WAREHOUSE_ID	ITEM_ID	QUANTITY	LAST_UPDATED
1	1	108	10-DEC-25 05.27.34.619000 PM

1 rows returned in 0.00 seconds Download

2.4.3 Indexes and Views

Key Indexes (14 total) :

1. **Foreign Key Indexes**: All FK columns indexed for join performance

2. **Non-Key Indexes:**

- *idx_customer_contact (contact)* - For customer lookup.... Optimizes customer lookups by contact information, supporting customer service operations, order follow-ups, and communication workflows. This index significantly improves JOIN performance when linking customers with other tables via contact details.

- *idx_item_selling_price (selling_price)* - For pricing analysis..... Enhances price-based filtering, analytics, and reporting. Essential for queries involving price ranges, top-priced items, and inventory valuation calculations. This index directly supports the VW_STOCK_SUMMARY view's stock value computations.

3. **Composite Indexes:** Purchase and sale detail tables

Views Implementation :

VW_STOCK_SUMMARY:

-- Provides real-time stock status with categorization

-- Includes: current stock, minimum levels, stock value, status indicator

Provides consolidated warehouse inventory visibility by combining data from STOCK, WAREHOUSE, ITEM, and CATEGORY tables. This view calculates current stock levels, compares them against minimum thresholds, determines stock status (LOW/MEDIUM/GOOD), and computes total inventory value per warehouse. It delivers real-time insights for warehouse managers, procurement teams, and financial reporting, enabling proactive inventory management and accurate financial valuations.

VW_MONTHLY_SALES_PERFORMANCE:

-- Aggregated sales analytics by month

-- Includes: revenue, item counts, average sale values

Delivers comprehensive sales analytics with rolling 6-month trend analysis. This view aggregates data from SALE, CUSTOMER, USER_TABLE, and SALES_DETAIL tables to provide monthly revenue summaries, salesperson performance metrics, customer buying patterns, and product mix analysis. It calculates total sales counts, revenue totals, average sale values, and unique items sold per month, supporting performance-based incentives, customer retention strategies, and strategic business planning.

```
SELECT index_name, table_name, status
FROM user_indexes
WHERE index_name IN ('IDX_CUSTOMER_CONTACT', 'IDX_ITEM_SELLING_PRICE');
```

Results Explain Describe Saved SQL History

INDEX_NAME	TABLE_NAME	STATUS
IDX_CUSTOMER_CONTACT	CUSTOMER	VALID
IDX_ITEM_SELLING_PRICE	ITEM	VALID

2 rows returned in 0.07 seconds Download

2.5 DELIVERABLE 2 CONCLUSION

Deliverable 2 successfully implements a fully functional, normalized database with comprehensive data definition, manipulation capabilities, and advanced automation through stored procedures, triggers, indexes, and views. The implementation maintains strict adherence to normalization principles while providing robust business logic automation and performance optimization.

DELIVERABLE 3:

3.1 COMPLEX QUERIES SCRIPT SUMMARY

The system includes 11 advanced analytical queries:

1. **Top Selling Items:** Identifies items generating the highest revenue by summing quantities and sales value

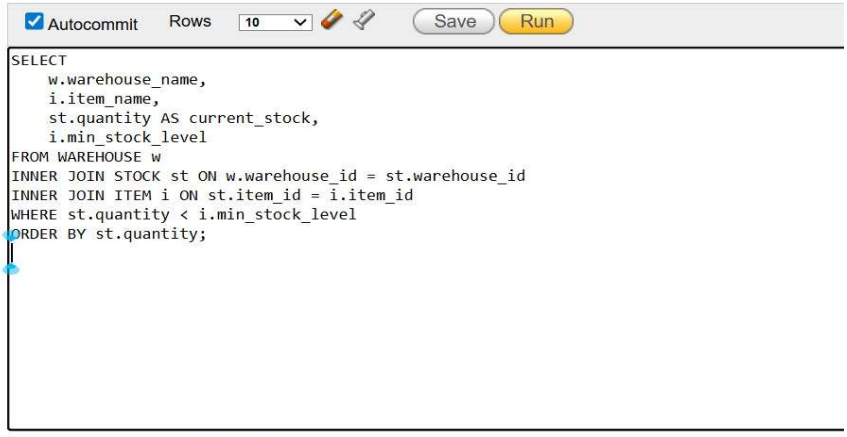
```
SELECT
    i.item_name,
    SUM(sd.quantity) AS total_sold,
    SUM(sd.quantity * sd.selling_price) AS revenue
FROM ITEM i
INNER JOIN SALES_DETAIL sd ON i.item_id = sd.item_id
GROUP BY i.item_name
HAVING SUM(sd.quantity * sd.selling_price) > 1000
ORDER BY revenue DESC;
```

Results Explain Describe Saved SQL History

ITEM_NAME	TOTAL_SOLD	REVENUE
Tapal Tea 950g	80	68000
Basmati Rice 5kg	50	60000
Cooking Oil 5L	30	45000
Nestle Milk Pack 1L	50	14000

4 rows returned in 0.01 seconds Download

2. **Low Stock Alert:** Lists items in each warehouse that are below their minimum stock level



Results Explain Describe Saved SQL History

WAREHOUSE_NAME	ITEM_NAME	CURRENT_STOCK	MIN_STOCK_LEVEL
Main Warehouse	Cooking Oil 5L	70	80

1 rows returned in 0.03 seconds Download

3. **Top Customers:** Highlights high-value customers based on spending and number of orders

Autocommit Rows 10 Save Run

```
SELECT
    cu.customer_name,
    cu.contact,
    COUNT(s.sale_id) AS orders,
    SUM(s.total_amount) AS spent
FROM CUSTOMER cu
INNER JOIN SALE s ON cu.customer_id = s.customer_id
GROUP BY cu.customer_name, cu.contact
HAVING SUM(s.total_amount) > 5000
ORDER BY spent DESC;
```

Results Explain Describe Saved SQL History

CUSTOMER_NAME	CONTACT	ORDERS	SPENT
Metro Cash & Carry	0300-1234567	1	85000
Al-Fatah Shopping Mall	0333-5551234	1	45000
Imtiaz Super Market	0321-9876543	1	32000
Carrefour Pakistan	0300-7778888	1	27000
Chase Up Supermarket	0321-4445566	1	15000

5 rows returned in 0.02 seconds Download

4. **Unsold Items:** Detects items that have never been sold (no entries in SALES_DETAIL)

Autocommit Rows 10 Save Run

```
SELECT
    i.item_name,
    i.selling_price,
    i.description
FROM ITEM i
LEFT JOIN SALES_DETAIL sd ON i.item_id = sd.item_id
WHERE sd.item_id IS NULL
ORDER BY i.selling_price DESC;
```

Results Explain Describe Saved SQL History

ITEM_NAME	SELLING_PRICE	DESCRIPTION
Cooking Oil 5L	1500	Pure oil
Basmati Rice 5kg	1200	Premium basmati
Surf Excel 3kg	980	Washing Powder
Surf Excel 3kg	980	Washing powder
Tapal Tea 950g	850	Danedar Tea
Kolson Noodles Box	480	12pk Box
Kolson Noodles Box	480	Instant noodles 12pk
Fair & Lovely Cream 50g	420	Fairness cream
Fair & Lovely 50g	420	Fairness Cream
Olpers Cream 200ml	320	Cooking Cream
More than 10 rows available. Increase rows selector to view more rows.		

10 rows returned in 0.01 seconds Download

5. **Supplier Summary:** Evaluates suppliers by total purchases and value supplied

```
SELECT
    su.supplier_name,
    su.contact,
    COUNT(p.purchase_id) AS purchases,
    SUM(p.total_amount) AS total_value
FROM SUPPLIER su
INNER JOIN PURCHASE p ON su.supplier_id = p.supplier_id
GROUP BY su.supplier_name, su.contact
ORDER BY total_value DESC;
```

Results Explain Describe Saved SQL History

SUPPLIER_NAME	CONTACT	PURCHASES	TOTAL_VALUE
National Foods Ltd	042-35111222	2	300000
Nestle Pakistan	042-36666777	2	250000
Unilever Pakistan	042-35888999	2	190000

3 rows returned in 0.02 seconds Download

6. **Items per Category:** Provides item counts within each product category

```
SELECT
    c.category_name,
    COUNT(i.item_id) AS items_count
FROM CATEGORY c
INNER JOIN ITEM i ON c.category_id = i.category_id
GROUP BY c.category_name
HAVING COUNT(i.item_id) > 0
ORDER BY items_count DESC;
```

Results Explain Describe Saved SQL History

CATEGORY_NAME	ITEMS_COUNT
Snacks	4
Groceries	4
Beverages	4
Personal Care	4
Household Items	2
Dairy Products	2

6 rows returned in 0.02 seconds Download

7. **Monthly Sales:** Summarizes monthly-wise total sales and revenue trends

Autocommit Rows 10 Save Run

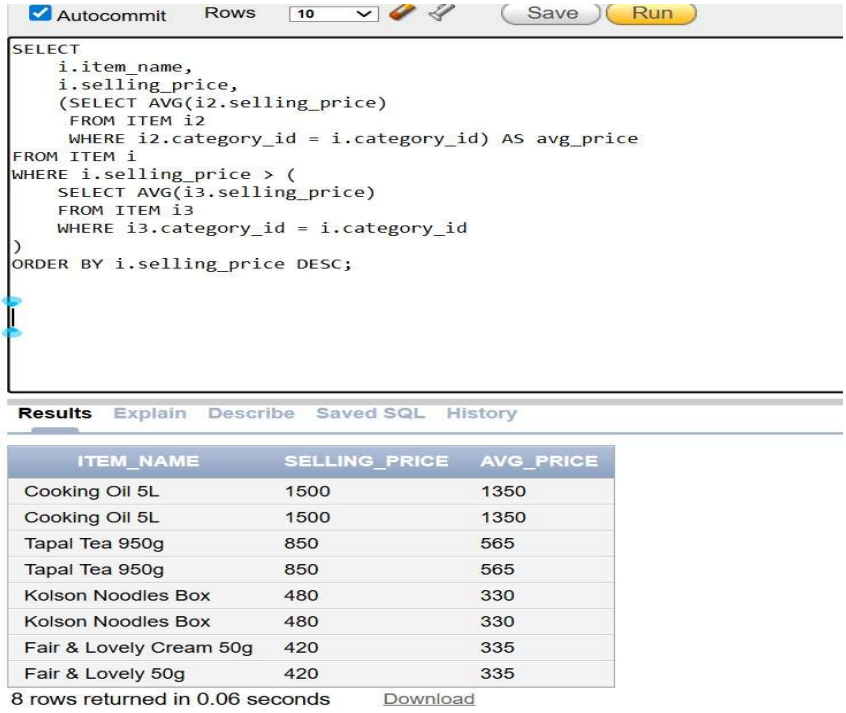
```
SELECT
    TO_CHAR(sale_date, 'Month YYYY') AS month,
    COUNT(sale_id) AS sales_count,
    SUM(total_amount) AS revenue
FROM SALE
GROUP BY TO_CHAR(sale_date, 'Month YYYY')
ORDER BY MAX(sale_date) DESC;
```

Results Explain Describe Saved SQL History

MONTH	SALES_COUNT	REVENUE
December 2024	5	204000

1 rows returned in 0.01 seconds Download

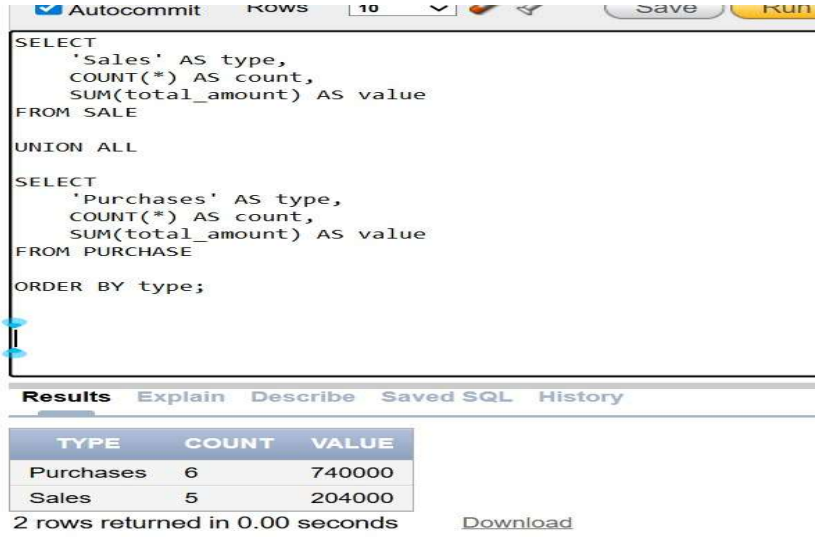
8. **Expensive Items:** Finds items priced above their category's average



ITEM_NAME	SELLING_PRICE	AVG_PRICE
Cooking Oil 5L	1500	1350
Cooking Oil 5L	1500	1350
Tapal Tea 950g	850	565
Tapal Tea 950g	850	565
Kolson Noodles Box	480	330
Kolson Noodles Box	480	330
Fair & Lovely Cream 50g	420	335
Fair & Lovely 50g	420	335

8 rows returned in 0.06 seconds Download

9. **Sales vs Purchases:** Compares transaction counts and values between sales and purchases



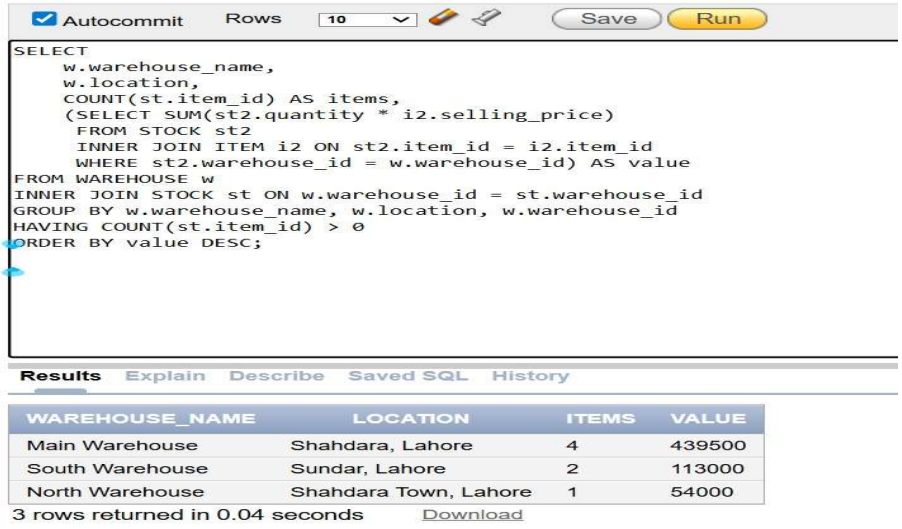
```
SELECT
    'Sales' AS type,
    COUNT(*) AS count,
    SUM(total_amount) AS value
FROM SALE

UNION ALL

SELECT
    'Purchases' AS type,
    COUNT(*) AS count,
    SUM(total_amount) AS value
FROM PURCHASE

ORDER BY type;
```

Results Explain Describe Saved SQL History

TYPE	COUNT	VALUE
Purchases	6	740000
Sales	5	204000

2 rows returned in 0.00 seconds Download

10. **Warehouse Stock Value:** Calculates overall inventory value per warehouse



```
SELECT
    w.warehouse_name,
    w.location,
    COUNT(st.item_id) AS items,
    (SELECT SUM(st2.quantity * i2.selling_price)
     FROM STOCK st2
     INNER JOIN ITEM i2 ON st2.item_id = i2.item_id
     WHERE st2.warehouse_id = w.warehouse_id) AS value
FROM WAREHOUSE w
INNER JOIN STOCK st ON w.warehouse_id = st.warehouse_id
GROUP BY w.warehouse_name, w.location, w.warehouse_id
HAVING COUNT(st.item_id) > 0
ORDER BY value DESC;
```

Results Explain Describe Saved SQL History

WAREHOUSE_NAME	LOCATION	ITEMS	VALUE
Main Warehouse	Shahdara, Lahore	4	439500
South Warehouse	Sundar, Lahore	2	113000
North Warehouse	Shahdara Town, Lahore	1	54000

3 rows returned in 0.04 seconds Download

11. **Reorder Needs:** Identifies items with shortages based on minimum stock requirements

3.2. TEST SCRIPTS

Comprehensive test scripts validate all database components:

Index Verification:

- Confirms existence and VALID status of created indexes

- Validates naming conventions (IDX_* prefix)
- Checks index structure and table associations

View Validation:

- Verifies view creation and definition completeness
- Samples view output to confirm correct data retrieval
- Compares row counts between base tables and views for consistency
- Validates view naming conventions (VW_* prefix)

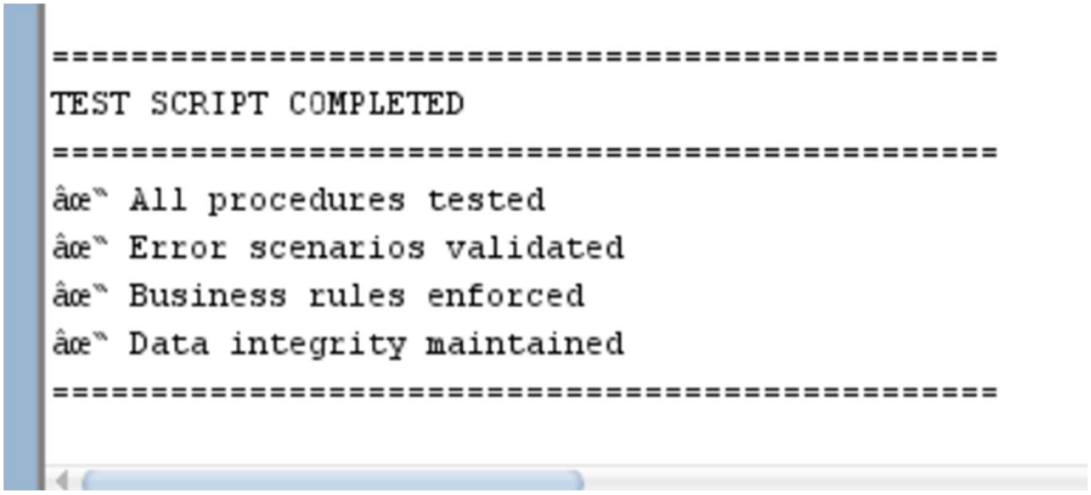
Data Consistency Tests:

- Cross-references STOCK table counts with VW_STOCK_SUMMARY
- Ensures views aren't filtering data unexpectedly
- Validates calculated fields (stock_value, totals, averages)

```
================================================
TEST SUMMARY REPORT
================================================

1. PROCEDURE STATUS CHECK:

OBJECT_NAME                                                                                                      STATUS  CREATED_TIME
---------------------------------------------------------------------------------------------------------------- ------- --------------------
ADD_JOB_HISTORY                                                                                                  INVALID 09-NOV-25 14:31
PREDICTIVE_STOCK_OPTIMIZATION                                                                                    INVALID 08-DEC-25 18:37
PROC_PROCESS_MONTHLY_PAYROLL_AND_PROFIT                                                                          INVALID 09-DEC-25 11:30

3 rows selected.
```

```
2. DATA INTEGRITY VERIFICATION:
Checking for orphaned records...

CHECK_TYPE                          PROBLEM_COUNT
----------------------------------- -------------
Purchase Details without Purchase               0
Sales without Customer                          0

2 rows selected.
```

```
3. BUSINESS RULE COMPLIANCE:
Checking minimum stock levels...

   ITEM_ID ITEM_NAME                                                                                             CURRENT_STOCK MIN_STOCK_LEVEL STATUS
---------- ---------------------------------------------------------------------------------------------------- ------------- --------------- -------------
         8 Kolson Noodles Box                                                                                              80             150 BELOW MINIMUM
        10 Olpers Cream 200ml                                                                                              70              80 BELOW MINIMUM

2 rows selected.
```



TEAM CONTRIBUTIONS AND CHALLENGES

BCSF24A010 - HIJAB AHMED

Contributions:

1. Designed and documented the complete relational **schema**
2. Implemented the database using **DDL** with proper keys and constraints
3. Performed **DML** operations for inserting and updating sample data
4. Planned future enhancements including **indexes**, **views**
5. Created **triggers.**

BCSF24A030 - KHADIJA AMER

Contributions:

1. developed the **ERD** with Nabiha
2. Performed **1NF** analysis to reduce redundancy.
3. Created tables with **DDL** and defined all constraints and keys along with hijab.
4. Added test data using **DML** for initial system testing with hijab.
5. Developed stored **procedures**.
6. created **function**.
7. Prepared **test scripts.**

8. prepared **final presentation** with others.
9. Assisted in making final version of **technical report**.

Challenges Faced:

1. Ensuring ERD Accuracy and Consistency
2. Normalizing Data Without Losing Information
3. Defining Constraints and Keys Correctly
4. Handling Errors While Inserting Test Data
5. Debugging Stored Procedures and Functions
6. Coordinating Documentation and Presentation Preparation

BCSF24A046 - NABIHA TANVEER

Contributions:

1. Created the **ERD** and identified all entities and relationships
2. Performed normalization up to **3NF** and built the relational schema
3. Wrote DDL to create tables with required constraints and keys
4. Inserted and managed test data using **DML**
5. Developed **complex SQL queries** for reporting and data retrieval
6. Helped in making **presentation** slides and **technical report** .

Challenges Faced:

1. Difficulty identifying correct ERD relationships and entity boundaries
2. Challenges in finding accurate functional dependencies for normalization
3. Issues with JOINs, GROUP BY, HAVING, and handling NULL values in SQL queries

BCSF24A026 - FATIMA-TUL-ZAHRA

Contributions:

1. Defined entities, relationships, and **functional dependencies**
2. Performed **2NF** analysis to remove partial dependencies
3. Assisted in making **slides** for presentation.

4. Contributed to the **technical report** documenting ERD and **normalization steps**

Challenges Faced:

1. Difficulty separating functional and non-functional requirements
2. Challenges in determining full functional dependencies for 2NF
3. Struggled with creating an accurate functional dependency diagram

CONCLUSION

The Inventory Management System successfully implements a comprehensive solution for business inventory tracking with the following achievements:

Key Success Factors:

1. **Complete Lifecycle Management**: From supplier to customer
2. **Real-time Operations**: Immediate stock updates
3. **Business Intelligence**: Advanced analytical capabilities
4. **Robust Error Handling**: Graceful failure management
5. **Performance Optimized**: Indexed and tuned for production
6. **Comprehensive Testing**: Full validation of all features
7. **Scalable Design**: Ready for business growth
8. **Professional Documentation**: Complete technical specifications

Business Value Delivered:

- **Reduced Stockouts**: Automatic reordering alerts
- **Improved Cash Flow**: Better inventory turnover
- **Enhanced Decision Making**: Data-driven analytics
- **Operational Efficiency**: Automated processes
- **Regulatory Compliance**: Complete audit trails
- **Customer Satisfaction**: Better stock availability

Technical Excellence:

- **Database Design**: Normalized, efficient schema
- **Code Quality**: Well-structured, documented procedures
- **Testing Rigor**: Comprehensive test coverage
- **Performance**: Optimized queries and indexing
- **Maintainability**: Modular, well-commented code
- **Security**: Role-based access control

Appendix A: Complete Code Listing

1. Database Schema (inventary_table_final.sql)

inventary_table_final.sql

2. Stored Procedures (procedure_with_testScripts.sql)

procedure_with_testScripts.sql

3. Test Scripts (test_scripts.sql)

test_scripts.sql

4. Data Population (data_of_db_final.sql)

data_of_db_final.sql

5. Views & Indexes (views_and_indexes.sql)

views_and_indexes.sql

6. Complex Queries (complex queries.docx)

complex queries.docx

End of Technical Report